

AI Response Analysis and Prompt Engineering Platform

Student Project Proposal

August 16, 2026

1 Higher-order goal

... secondary framing

This project aims to help users understand, evaluate, and improve the AI responses they receive from generative AI systems. The proposed software will primarily function as an AI response analysis and explanation platform, with prompt engineering used as a supporting mechanism for improving unsatisfactory responses.

The immediate engineering objective is to build a usable dashboard that examines the quality, structure, relevance, clarity, completeness, and constraint alignment of AI-generated responses. It will also provide user-facing explanations connecting observable response characteristics with the user's prompt and requested requirements. The project will not attempt to reveal hidden model reasoning or internal chain-of-thought.

2 Time-to-value

... primary heuristic

- The first iteration will provide AI response generation and a basic response quality analysis.
- Subsequent iterations will add response explanation, response diagnosis, response comparison, prompt analysis, and prompt improvement.
- Development will prioritize simple, demonstrable features that can be tested independently and integrated into the final dashboard.
- Success is defined by helping users understand why an AI response has its observable characteristics, identify problems, and improve the result when necessary.

3 Problem Statement

Users frequently receive AI responses that are too long, too short, too technical, incomplete, poorly structured, irrelevant, or not aligned with their requested format. Although users can see the final response, they may not understand what caused these observable characteristics or how to decide whether the response is actually suitable for their goal.

Common pain points include:

- **Limited response understanding:** users may receive an answer without knowing whether it is relevant, complete, clear, concise, well-structured, or aligned with their request.
- **Difficulty diagnosing poor responses:** users may know that an answer is unsatisfactory but may not know whether the problem comes from missing context, excessive detail, weak structure, or unmet constraints.

- **Lack of response transparency:** users generally see the final answer but do not receive a simple analysis of the observable factors associated with that answer.
- **Repeated trial and error:** users may repeatedly regenerate answers instead of understanding what should be changed.
- **Inefficient AI usage:** users with limited messages, responses, or usage quotas can benefit from analyzing an answer and improving the next request before using another interaction.

Why this matters now (contextual relevance): Generative AI is being used for education, programming, research, writing, summarization, coding, and everyday information tasks. As AI-generated content becomes more common, users need practical ways to evaluate whether a response is useful and understand why it has a particular style, level of detail, or structure. This is especially valuable when users have limited AI interactions available and need to obtain useful results efficiently.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **web-based AI Response Analysis and Prompt Engineering Platform** with the following components:

- **AI response analyzer:** evaluates relevance, clarity, conciseness, completeness, structure, usefulness, and adherence to requested constraints.
- **Response diagnosis:** identifies observable strengths and weaknesses such as excessive length, missing information, poor structure, unnecessary detail, or failure to follow requested format.
- **Response explanation:** provides a user-facing explanation of why the response has its observable characteristics by relating the response to prompt instructions and requested requirements.
- **Prompt analyzer:** identifies task, topic, audience, context, constraints, desired length, tone, and output format.
- **Prompt optimizer:** suggests changes to the prompt when the response does not satisfy the user's objective.
- **Comparison dashboard:** compares responses from different prompt versions and shows how their observable characteristics differ.

4.2 Core workflow

1. User enters a prompt and generates an AI response.
2. The system analyzes the response for relevance, clarity, conciseness, completeness, structure, and adherence to the user's requirements.
3. The dashboard identifies strengths, weaknesses, and possible response problems.
4. The response explanation relates observable characteristics of the answer to identifiable instructions in the prompt.
5. If the response is unsatisfactory, the system analyzes the prompt and suggests targeted improvements.

6. The user can generate a revised response and compare it with the original response.
7. The dashboard presents the comparison as a learning loop showing how prompt changes can affect observable response characteristics.

4.3 Operational constraints for an educational setting

- Keep the project focused on text-based generative AI response analysis and user-facing explanation, with prompt engineering as a supporting improvement feature.
- Use an existing AI API instead of developing or training a large language model.
- Avoid claiming that individual words causally determine specific model outputs; explanations will be presented as likely effects based on prompt structure and observable response changes.
- Keep the first implementation small enough for a three-person team to complete within one semester.
- Prioritize software correctness, usability, measurable response analysis, response explanation, and practical experimentation over advanced AI research.

5 Solution Approach

... Engineering focus

This is an engineering course project, so the focus will be on building a practical software platform around an existing generative AI service. The main contribution is the integration of response analysis, response diagnosis, user-facing explanation, prompt analysis, and response comparison into one application.

5.1 Response analysis and prompt assistance

... non-research

Response analysis will be the primary analysis component of the system. It will examine observable characteristics of generated answers, including:

- **Relevance:** whether the response addresses the user's actual request and avoids unnecessary information.
- **Clarity:** whether the response is understandable and uses an appropriate level of language for the requested audience.
- **Conciseness:** whether the response avoids unnecessary repetition and matches the requested level of detail.
- **Completeness:** whether the important parts of the requested task have been addressed.
- **Structure:** whether headings, bullets, steps, examples, or other requested formats are used appropriately.
- **Constraint adherence:** whether the response follows requirements such as word limits, tone, format, or audience.
- **Response diagnosis:** identifies observable problems and gives practical suggestions for improvement.

The response explanation will then connect these observations with the prompt. For example, the system may identify that a response became detailed because the prompt requested a detailed explanation, or that the response was difficult for a beginner because no audience or complexity level was specified. These are observable, user-facing explanations rather than claims about hidden model reasoning.

Prompt assistance will identify task, topic, audience, context, constraints, length, tone, and output format, and can suggest improvements when the response analysis shows that the original request was insufficient.

5.2 Web architecture

...web-based solution

A typical 3-tier web architecture:

- **Frontend:** responsive interface for entering prompts, viewing AI responses, displaying response-quality scores, showing response explanations and detected issues, and comparing responses.
- **Backend API:** handles AI API requests, response analysis, response diagnosis, response explanation, prompt analysis, prompt optimization, comparison logic, and history.
- **Database:** stores prompts, prompt versions, AI responses, selected objectives, analysis results, and timestamps.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build the application and run tests on every change.
- Validate backend API and core prompt-processing functionality.
- Produce repeatable deployment artifacts.
- Support incremental development and safe integration of features.

6 Evaluation Criterion

...measurable and attributable

We define success primarily using measurable criteria related to AI response quality, response alignment, explanation usefulness, and the ability of the platform to help users diagnose and improve unsatisfactory responses.

6.1 Primary evaluation metric

AI response quality and alignment: measure whether the platform can consistently identify important observable response characteristics and whether generated responses satisfy the user's requested objective and constraints.

- Evaluate response relevance, clarity, conciseness, completeness, structure, and compliance with requested constraints.
- Check whether identified response strengths and weaknesses are consistent with the actual response.
- Check whether the response explanation is supported by observable prompt and response characteristics.
- Compare original and revised responses to determine whether a targeted prompt change improved the selected response criteria.

6.2 Secondary evaluation metrics

- **Response quality:** relevance, clarity, conciseness, completeness, structure, and usefulness.
- **Response alignment:** degree to which the response follows requested length, format, audience, tone, and other constraints.
- **Response diagnosis accuracy:** whether detected strengths, weaknesses, and constraint mismatches correspond to the actual response.
- **Explanation usefulness:** whether users understand the observable reasons associated with the response characteristics.
- **Prompt impact:** whether targeted prompt changes produce measurable changes in response characteristics.
- **Efficiency:** reduction in unnecessary repeated prompts needed to obtain a satisfactory response during controlled tests.

6.3 Pilot validation plan

... high-level

- Prepare common tasks such as explanation, summarization, coding, question answering, and summarizing text.
- Generate representative AI responses and evaluate them using the response-analysis dashboard.
- Test whether the system correctly identifies issues such as excessive length, missing information, poor structure, or unmet format requirements.
- Ask a small group of users whether the response explanations are understandable and whether the diagnosis helps them improve an answer.
- Test selected prompt changes and compare the resulting responses to determine whether the platform identifies meaningful improvements.

7 Scalability

... with foresight

The platform should be scalable in both its software architecture and its ability to support different AI response types, evaluation criteria, and improvement workflows.

1. **Scalable (theoretically):** the system can support growth by:
 - separating frontend, backend, AI integration, and database components,
 - using pagination for conversation and prompt history,
 - indexing common database queries,
 - allowing additional response metrics, explanation criteria, and prompt objectives to be added as modules,
 - supporting multiple AI providers through a common backend interface in future versions.
2. **Operationally deployable (practically):** the system should run using:
 - standard web hosting,
 - a managed or local database,
 - containerized deployment where appropriate,
 - CI/CD for repeatable builds and releases.

8 Engine availability heuristic

A mature set of tools and services is available to implement the platform:

- Web framework such as React or Next.js for the frontend.
- Python and FastAPI for backend REST APIs.
- A generative AI API for response generation.
- Natural-language processing and text-processing libraries for prompt and response analysis.
- PostgreSQL or another suitable database for prompt and response history.
- Visualization libraries for quality scores and prompt comparisons.
- Testing tools for backend, API, and prompt-analysis validation.
- Docker and CI/CD tools for deployment and repeatable development.

We will use mature components to focus on AI response analysis, response explanation, prompt improvement, usability, and system integration rather than developing a language model or AI infrastructure from scratch.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Prompt input and AI response generation.
- Core response-quality analysis for relevance, clarity, conciseness, completeness, structure, and constraint adherence.
- Response strengths, weaknesses, and response-diagnosis display.
- Basic response explanation connecting observable characteristics with prompt requirements.
- Basic prompt component identification.
- Initial prompt improvement suggestions.
- Database storage for prompts and responses.
- Basic frontend and backend integration.
- Automated tests for the main workflow.

9.2 Subsequent deliverables

- Detailed response explanation and response diagnosis.
- Prompt and response comparison with quality metrics.
- Response analysis for multiple task types such as explanation, coding, summarization, and question answering.
- Conversation and response history.
- Prompt evolution showing how successive changes affect responses.

- Prompt A/B testing for selected use cases.
- Prompt templates and learning suggestions.
- Optional support for multiple AI providers if time permits.

10 Risks and mitigations

- **Overclaiming AI explanations:** do not claim access to hidden model reasoning; explain only observable relationships between prompt structure and response characteristics.
- **Subjective response quality:** use clearly defined metrics and controlled comparisons rather than claiming a single universal quality score.
- **API limitations or cost:** use controlled test cases, configurable models, and minimize unnecessary API calls.
- **Project scope becoming too large:** prioritize response analysis, response explanation, response comparison, and basic prompt improvement before optional features.
- **Data privacy:** avoid storing unnecessary personal or sensitive conversation content and provide clear controls for stored history.
- **Model variability:** record the selected model and relevant request information when comparing responses so that results can be interpreted correctly.
- **Evaluation ambiguity:** define test prompts, objectives, response criteria, and comparison procedures before final evaluation.

11 Summary

This project proposes a deployable web platform centered on understanding and improving AI-generated responses. The system evaluates responses for relevance, clarity, conciseness, completeness, structure, usefulness, and adherence to requested constraints. It then provides a user-facing diagnosis and explanation of observable response characteristics.

Prompt engineering is used as a supporting mechanism: the system identifies important prompt components and suggests targeted changes when the response does not satisfy the user's goal. Users can compare original and revised responses to see whether those changes improved the result. The overall learning loop is:

Prompt -> AI Response -> Response Analysis -> Explanation -> Prompt Improvement -> Revised Response -> Comparison

The project remains focused on software engineering rather than developing a new language model. It combines frontend development, backend APIs, AI integration, natural-language processing, database management, visualization, testing, and deployment into a manageable three-person semester project.